```
$ python -m timeit -s 'text = "my name is david. i fight
goliath."; char = "s"'  'text.find(char)'
10000000 loops, best of 3: 0.196 usec per loop
```

Even though micro-benchmarking as a concept might look very interesting, more often than not these timings might be misleading, since a large piece of code would be doing a lot more in the background. Operations like caching, indexing, parallel computing, memory and processor optimizations are not accounted for during micro-benchmarking and hence such benchmarks might not be an accurate representation of code performance.

Regular Expressions

Regular Expression, also known as regex is a powerful tool that enables us to perform quick match and replace operations on strings. Every regex is a sequence of character(s) which identifies patterns in a string using characters and metacharacters. In Python we have a re module that helps with Regular Expressions and provides operations that are similar to those found in Perl.

Before we jump on to the regex bandwagon, we should keep note of the fact that regular expressions use '\' to escape any metacharacters in a regex string. At the same time string literals in python use an identical escaping strategy which leads to confusion. For example, to match a backslash ('\') character, the regex would be (\\). We would need to escape individual backslashes in the python string. The resultant pattern would look something like this "\\\\", when represented as a string literal. The aforementioned problem is solved using raw string notation available in python, wherein metacharacters are not handled in a special way. These strings are prefixed using "r", hence r"\t" is a two-character string with no special meaning while "\t" is a tab space. We would be using raw strings as regular expressions henceforth.

re.search() & re.match()

The re module essentially provides two primitive operations, one is a matching operation while the other is a searching operation. At the bare bones, both the operations lookout for patterns in the target string, but a matching operation considers a match only at the starting of the string while a searching operation will consider a match anywhere in the string.